Vibe Coding: A Complete Beginner-to-Production Guide

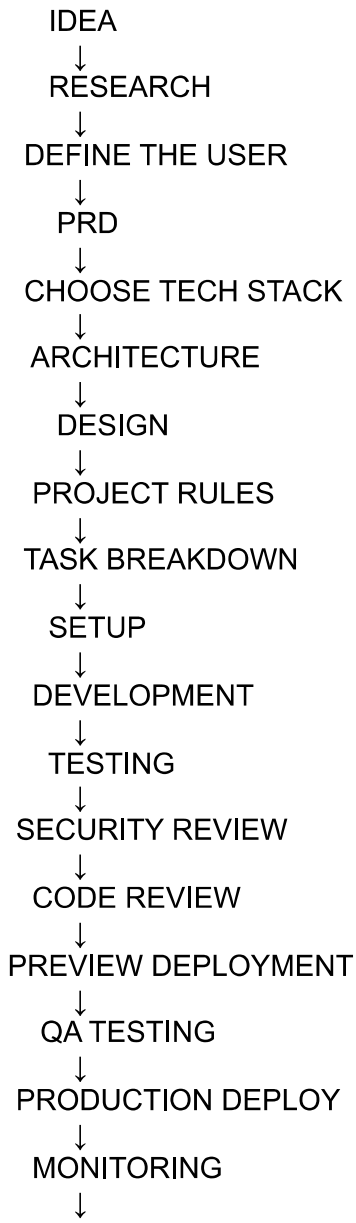
Vibe Coding: A Complete Beginner-to-Production Guide

From Idea → Planning → AI Coding → Testing → Deployment → Production

---

1. The Complete Vibe Coding Workflow

Before writing any code, understand the entire process.



2. Step 1: Define What You Want to Build

Before opening your AI coding tool, answer five questions.

1. What problem are you solving?

Example:

> College students struggle to organize assignments, notes and deadlines.

2. Who is the user?

Example:

> BCA, BSc CS and BTech students.

3. What is the main outcome?

Example:

> Give students one place to manage their academic work.

4. What is the MVP?

MVP means **Minimum Viable Product.**

For example:

Authentication
Dashboard
Notes
Assignments
Deadlines

Don't start with:

AI Tutor
Community

Vibe Coding: A Complete Beginner-to-Production Guide

Social Feed

Build the core product first.

5. What is NOT part of the first version?

This is equally important.

Example:

Out of scope:
- Mobile application
- Payments
- AI chatbot
- Social features

This prevents AI from continuously expanding the project.

---

3. Step 2: Research Before Coding

Don't ask AI to build an idea that you haven't researched.

Research:

Users

- Who needs this?
- What are their problems?
- What alternatives already exist?

Competitors

Look at:

- Features

Vibe Coding: A Complete Beginner-to-Production Guide

- Pricing
- User experience
- Strengths
- Weaknesses

Technical feasibility

Check:

- APIs
- Authentication
- Database requirements
- Third-party services
- Hosting
- Cost

AI tools can help with research

You can use:

- ChatGPT
- Perplexity
- Google
- Official documentation
- GitHub

For technical decisions, prefer **official documentation** over random blog posts.

---

4. Step 3: Choose Your AI Coding Tool

Don't use the same tool for every project.

For beginners

Replit

Good when you want:

- Browser-based development
- Minimal setup
- Quick prototypes
- Full-stack experimentation

Lovable

Good for:

- Web apps
- SaaS prototypes
- UI-heavy applications

Vibe Coding: A Complete Beginner-to-Production Guide

Bolt

Good for:

- Fast web application prototypes
- JavaScript/TypeScript projects
- Browser-based development

Cursor

Good when you want:

- More control
- Existing repositories
- Larger projects
- Local development
- Professional development workflows

Claude Code

Good for:

- Terminal-based development
- Larger codebases
- Repository-level tasks
- Agentic development workflows

Vibe Coding: A Complete Beginner-to-Production Guide

6. Recommended Beginner Stack

If you're serious about learning development through vibe coding, this is a good starting stack:

AI IDE
Cursor

Frontend
Next.js

Language
TypeScript

Styling
Tailwind CSS

Database
PostgreSQL / Supabase

Authentication
Supabase Auth

Version Control
Git + GitHub

Testing
Playwright

Deployment
Vercel

7. Step 4: Install the Development Environment

For a typical web project, install:

Required

1. Cursor or VS Code
2. Git
3. Node.js LTS
4. npm
5. GitHub account
6. Modern web browser

Optional

Docker
GitHub CLI
Postman / Bruno
Database client
Vercel CLI

Vibe Coding: A Complete Beginner-to-Production Guide

node --version
npm --version
git --version

---

8. Step 5: Create Your Project

Create a project folder:

mkdir student-dashboard
cd student-dashboard

Initialize Git:

git init

Then create a GitHub repository.

From the beginning, your project should have version control.

Why?

Because AI can break your project.

Git gives you the ability to:

Change
↓
Test
↓
Break something
↓
Compare
↓
Rollback

---

9. Step 6: Create Your Project Documentation

This is one of the most important parts of structured vibe coding.

Before asking AI to build features, create your project context.

Vibe Coding: A Complete Beginner-to-Production Guide

```
project/
│
├── docs/
│   ├── PRD.md
│   ├── ARCHITECTURE.md
│   ├── DESIGN.md
│   ├── TEST_PLAN.md
│   ├── SECURITY.md
│   ├── DECISIONS.md
│   └── MEMORY.md
│
├── .cursor/
│   └── rules/
│
├── src/
├── tests/
│
├── README.md
├── TASKS.md
├── .env.example
└── .gitignore
```

You don't need all of these for a tiny project.

For a beginner project, start with:

PRD.md
ARCHITECTURE.md
DESIGN.md
RULES.md
TASKS.md
README.md
.env.example

---

10. Step 7: Create `PRD.md`

What is PRD?

PRD = Product Requirements Document

It defines:

> **What are we building and why?**

Example:

Vibe Coding: A Complete Beginner-to-Production Guide

StudentHub

Problem
Students struggle to manage their academic notes, assignments and deadlines.

Target Users
BCA, BSc CS and BTech students.

Goal
Create a centralized academic productivity dashboard.

Core Features

1. Authentication
2. Dashboard
3. Notes
4. Assignments
5. Deadlines

MVP

- Signup
- Login
- Dashboard
- Create notes
- Edit notes
- Delete notes
- Create assignments

Out of Scope

- Payments
- AI tutor
- Mobile app
- Social feed

Success Criteria

A user should be able to:

1. Create an account
2. Login
3. Create a note
4. Edit a note
5. Delete a note
6. Create an assignment
7. Mark an assignment complete

Remember:

PRD = WHAT + WHY

Vibe Coding: A Complete Beginner-to-Production Guide

This defines:

How will the application work?

Example:

Architecture

Frontend
Next.js + TypeScript

Styling
Tailwind CSS

Backend
Next.js server-side functionality

Database
Supabase PostgreSQL

Authentication
Supabase Auth

Deployment
Vercel

Architecture

User
↓
Next.js UI
↓
Server Action / API
↓
Supabase
↓
PostgreSQL

Define your folder structure too:

```
src/
├── app/
├── components/
├── features/
├── services/
├── lib/
├── types/
└── utils/
```

Also define architectural rules.

Vibe Coding: A Complete Beginner-to-Production Guide

- UI components should not contain database logic.
- Database operations belong in services.
- Authentication must be verified server-side.
- Reusable UI should be placed in components.
- Business logic should remain separate from UI.

Remember:

Architecture = HOW

---

12. Step 9: Create DESIGN.md

This gives AI a consistent visual system.

Without it, you may get:

Page 1 → rounded cards

Page 2 → square cards

Page 3 → different buttons

Page 4 → completely different typography

Define:

```
# Design System

## Style
Modern
Minimal
Professional

## Typography
Inter

## Colors
Primary: #6366F1
Background: #F8FAFC
Text: #0F172A
Muted: #64748B

## Buttons

Primary
Secondary
Destructive

## Cards
```

- Mobile responsive
- Loading states
- Empty states
- Error states
- Accessible forms

Remember:

DESIGN.md = HOW IT SHOULD LOOK AND FEEL

---

13. Step 10: Create `RULES.md`

This is your project's **AI rulebook**.

Example:

```
# Development Rules

## General

- Use TypeScript.
- Reuse existing components.
- Do not duplicate logic.
- Keep functions small.
- Do not modify unrelated files.

## Before Coding

- Read the relevant project documentation.
- Inspect existing implementation.
- Reuse existing functionality where possible.
- Make a plan for large changes.

## UI

- Follow DESIGN.md.
- Maintain responsive design.
- Include loading states.
- Include error states.
- Include empty states.

## Security

- Never expose API keys.
- Validate user input.
- Verify authorization server-side.

## Testing
```

Vibe Coding: A Complete Beginner-to-Production Guide

Git

- Make small commits.
- Use descriptive commit messages.

For Cursor, these rules can also be organized inside:

.cursor/rules/

For example:

```
.cursor/
└── rules/
    ├── general.mdc
    ├── frontend.mdc
    ├── backend.mdc
    └── testing.mdc
```

---

14. Step 11: Create `TASKS.md`

Never ask AI to build the entire application in one prompt.

Break the project into tasks.

Tasks

Phase 1: Setup

- [ ] Initialize project
- [ ] Configure TypeScript
- [ ] Configure Tailwind
- [ ] Configure Git

Phase 2: Authentication

- [ ] Create signup page
- [ ] Create login page
- [ ] Configure authentication
- [ ] Protect dashboard
- [ ] Test authentication

Phase 3: Notes

- [ ] Create database table
- [ ] Create notes service
- [ ] Create notes UI
- [ ] Create note form
- [ ] Add edit functionality
- [ ] Add delete functionality

Vibe Coding: A Complete Beginner-to-Production Guide

TASK-001
↓
Implement
↓
Test
↓
Review
↓
Mark complete
↓
TASK-002

---

15. Step 12: Create `DECISIONS.md`

This stores important technical decisions.

Example:

```
# Architecture Decisions

## ADR-001

Decision:
Use Supabase for the database.

Reason:
Provides PostgreSQL, authentication and backend
services without managing our own infrastructure.

## ADR-002

Decision:
Use Next.js instead of React + Express.

Reason:
The application does not require a separate backend
and we want a unified full-stack application.
```

This prevents AI from randomly changing architectural decisions later.

---

16. Step 13: Create `MEMORY.md`

Think of this as the project's current state.

Vibe Coding: A Complete Beginner-to-Production Guide

Project Memory

Current Status

Authentication completed.

Notes feature in progress.

Completed

- Project setup
- Git setup
- Database setup
- Authentication

Current Task

TASK-012

Known Issues

- Mobile navbar needs improvement.
- Notes loading state is incomplete.

Next Step

Complete note creation.

A useful distinction is:

DECISIONS.md
Permanent decisions

MEMORY.md
Current project state

---

17. Step 14: Create `TEST_PLAN.md`

Define what "working" actually means.

Test Plan

Authentication

- User can signup
- User can login
- Invalid credentials show error
- Logged-out users cannot access dashboard

Notes

- User can create note
- User can edit note

Vibe Coding: A Complete Beginner-to-Production Guide

Responsive

Test at:

- 375px
- 768px
- 1440px

This becomes your testing checklist later.

---

18. Step 15: Create `SECURITY.md`

For production projects:

Security Requirements

Authentication

Private routes require authentication.

Authorization

Users can only access resources they own.

Secrets

Never expose secrets to client-side code.

Database

Use appropriate access policies.

Input

Validate all user input.

APIs

Validate request body and parameters.

File Uploads

Validate:

- File type
- File size
- Filename

Security should not be added five minutes before deployment.

Vibe Coding: A Complete Beginner-to-Production Guide

Never put real API keys inside your source code.

Instead:

DATABASE_URL=
SUPABASE_URL=
SUPABASE_ANON_KEY=
OPENAI_API_KEY=
STRIPE_SECRET_KEY=

Then your local environment contains the real values:

.env.local

Your repository should contain:

.env.example

not your actual secrets.

---

20. Step 17: Give the AI Project Context

Now open your AI coding tool.

Don't immediately say:

> Build my app.

Start with:

Read the following files before making any changes:

PRD.md
ARCHITECTURE.md
DESIGN.md
RULES.md
TASKS.md

Do not modify anything yet.

First:
1. Understand the product.
2. Understand the architecture.
3. Review the design system.

Vibe Coding: A Complete Beginner-to-Production Guide

7. Explain the implementation plan for TASK-001.

Do not write code yet.

This lets the AI understand the project before touching it.

---

21. Step 18: Build One Feature at a Time

Use this workflow:

Understand
↓
Plan
↓
Implement
↓
Test
↓
Review
↓
Commit

For example:

TASK-001
Create signup page

Then:

TASK-002
Implement signup logic

Then:

TASK-003
Add validation

Then:

TASK-004
Add authentication tests

---

22. Step 19: Use Vertical Slices

Instead of building:

Vibe Coding: A Complete Beginner-to-Production Guide

↓
Database

Build a complete user flow.

Example:

Signup
↓
Login
↓
Dashboard
↓
Create Note
↓
Save Note
↓
Display Note

Then:

Edit Note
↓
Save
↓
Display updated note

Then:

Delete Note
↓
Confirm
↓
Database update
↓
UI update

This makes debugging much easier.

---

23. Step 20: Use a Structured Prompt

A good coding prompt has six parts:

CONTEXT
TASK
FILES
CONSTRAINTS
ACCEPTANCE CRITERIA
TESTING

Vibe Coding: A Complete Beginner-to-Production Guide

CONTEXT

We are building a student notes application.
Read PRD.md, ARCHITECTURE.md and RULES.md.

TASK

Implement note creation.

FILES

Relevant files:
src/features/notes/
src/services/
src/types/

CONSTRAINTS

- Follow the existing architecture.
- Reuse existing components.
- Do not create a second database layer.
- Do not modify unrelated files.
- Validate input.

ACCEPTANCE CRITERIA

- Logged-in user can create a note.
- Title is required.
- Content is required.
- Invalid input shows an error.
- Note is saved to the database.
- New note appears in the UI.

TESTING

Add appropriate tests.
Run lint.
Run type checking.
Run relevant tests.

After implementation, report:

1. Files changed
2. What was implemented
3. Tests executed
4. Remaining issues

---

24. Step 21: Never Give AI Huge Tasks

Bad

Build the entire SaaS application.

Vibe Coding: A Complete Beginner-to-Production Guide

Build authentication.

Best

Create the signup UI.

Do not implement authentication yet.

Follow DESIGN.md.

Requirements:
- Email field
- Password field
- Confirm password field
- Validation
- Loading state
- Error state
- Mobile responsive

Do not modify unrelated files.

Small tasks give AI less room to misunderstand your project.

---

25. Step 22: Test Every Feature

After implementing a feature:

Code
↓
Lint
↓
Type check
↓
Unit tests
↓
Integration tests
↓
E2E tests

Type check

npm run typecheck

Lint

npm run lint

Tests

npm test

Vibe Coding: A Complete Beginner-to-Production Guide

~~npm run build~~

Your exact commands depend on the project's configuration.

---

26. Step 23: Use End-to-End Testing

E2E testing checks the application from the user's perspective.

For example:

Open website
↓
Signup
↓
Login
↓
Create note
↓
Refresh
↓
Verify note exists
↓
Edit note
↓
Delete note
↓
Logout

Tools such as Playwright are useful for automating browser-based tests.

---

27. Step 24: Review AI-Generated Code

Don't assume:

> "It compiled, so it's correct."

Ask the AI:

Review the implementation against:

PRD.md
ARCHITECTURE.md

Vibe Coding: A Complete Beginner-to-Production Guide

SECURITY.md

Check:

- Correctness
- Architecture
- Security
- Error handling
- Accessibility
- Responsive design
- Performance
- Code duplication

Do not modify anything yet.

Report all issues first.

Then fix them one by one.

---

28. Step 25: Learn How to Debug With AI

When something breaks, don't say:

> Fix this.

Give the AI structured information.

ERROR

[paste error]

EXPECTED BEHAVIOR

The user should be redirected to the dashboard.

ACTUAL BEHAVIOR

The page shows a 500 error.

STEPS TO REPRODUCE

1. Login
2. Click Dashboard
3. Error appears

CONSTRAINT

Do not change the database schema.

Vibe Coding: A Complete Beginner-to-Production Guide

Do not modify code yet.

Find the root cause.

Explain:
1. What is failing?
2. Why is it failing?
3. Which file is responsible?
4. What is the smallest fix?
5. How will we test the fix?

Then:

Implement the smallest fix.

Do not refactor unrelated code.

Run the relevant tests.

---

29. Step 26: Use Git Throughout Development

After a working feature:

git status

Then:

git add .

Then:

git commit -m "feat: add note creation"

Then:

git push

Use branches for larger projects:

```
main
│
├── feature/auth
├── feature/notes
├── feature/search
└── fix/mobile-navbar
```

This protects your working code from AI-generated mistakes.

Vibe Coding: A Complete Beginner-to-Production Guide

30. Step 27: Prepare for Deployment

Before deployment, verify:

Functionality

[ ] Signup
[ ] Login
[ ] Logout
[ ] CRUD operations
[ ] Forms
[ ] Error handling
[ ] Loading states
[ ] Empty states

UI

[ ] Mobile
[ ] Tablet
[ ] Desktop
[ ] Accessibility
[ ] Keyboard navigation

Security

[ ] No secrets in Git
[ ] Authentication verified
[ ] Authorization verified
[ ] Database security configured
[ ] Input validation
[ ] API validation

Code

[ ] TypeScript passes
[ ] Lint passes
[ ] Tests pass
[ ] Production build passes

---

31. Step 28: Deploy to Preview First

Your deployment pipeline should look like:

LOCAL
↓
PREVIEW
↓
QA
↓
PRODUCTION

Vibe Coding: A Complete Beginner-to-Production Guide

For example:

feature/notes
↓
Preview deployment
↓
Test
↓
Fix
↓
Merge
↓
Production

---

32. Step 29: Deploy to Production

For a Next.js application, Vercel is one common deployment option.

The general workflow is:

GitHub repository
↓
Connect to Vercel
↓
Configure environment variables
↓
Deploy
↓
Preview URL
↓
Test
↓
Production

You can also use the Vercel CLI when appropriate.

vercel

After testing:

vercel --prod

Vibe Coding: A Complete Beginner-to-Production Guide

33. Step 30: Configure Environment Variables

Your local environment might contain:

.env.local

Production should have its own values.

For example:

Development
DATABASE_URL=development-db

Preview
DATABASE_URL=staging-db

Production
DATABASE_URL=production-db

Don't assume your local environment is identical to production.

---

34. Step 31: Production QA

After deployment, test the actual live application.

Don't only test localhost.

Check:

Live URL
↓
Signup
↓
Login
↓
Core features
↓
Database operations
↓
Error handling
↓
Mobile
↓
Desktop

Also test:

Vibe Coding: A Complete Beginner-to-Production Guide

- Direct URLs
- Logged-out access
- Invalid inputs
- Slow network
- Empty database
- Wrong credentials

---

35. Step 32: Monitor the Application

Deployment isn't the end.

The production lifecycle is:

Deploy
↓
Monitor
↓
Collect feedback
↓
Find bugs
↓
Fix
↓
Test
↓
Deploy again

Consider adding:

- Error tracking
- Analytics
- Performance monitoring

Vibe Coding: A Complete Beginner-to-Production Guide

- Uptime monitoring

---

36. Step 33: Maintain Project Documentation

As the project changes, update:

TASKS.md
MEMORY.md
DECISIONS.md
README.md
ARCHITECTURE.md

Don't let documentation describe an application that no longer exists.

The AI needs accurate context.

---

37. The Recommended Project Structure

For a serious beginner project:

```
student-dashboard/
│
├── docs/
│   │
│   ├── PRD.md
│   ├── ARCHITECTURE.md
│   ├── DESIGN.md
│   ├── TEST_PLAN.md
│   ├── SECURITY.md
│   ├── DECISIONS.md
│   └── MEMORY.md
│
├── .cursor/
│   └── rules/
│       ├── general.mdc
│       ├── frontend.mdc
│       ├── backend.mdc
│       └── testing.mdc
│
├── src/
│   ├── app/
│   ├── components/
```

Vibe Coding: A Complete Beginner-to-Production Guide

```
    ├── types/
    └── utils/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── public/
├── .env.example
├── .gitignore
├── README.md
├── TASKS.md
├── package.json
└── ...
```

---

38. What Each File Does

File	Purpose	Stage
`PRD.md`	What are we building?	Planning
`ARCHITECTURE.md`	How will we build it?	Planning
`DESIGN.md`	How should it look?	Planning
`RULES.md`	How should AI code?	Planning
`TASKS.md`	What should we build next?	Development
`DECISIONS.md`	Why did we make this decision?	Development
`MEMORY.md`	What is the current project state?	Development
`TEST_PLAN.md`	How do we verify it?	Testing
`SECURITY.md`	How do we protect it?	Development
`.env.example`	What configuration is required?	Setup
`README.md`	How do humans use the project?	Documentation

Vibe Coding: A Complete Beginner-to-Production Guide

39. The Minimum Setup for Beginners

If all of this feels overwhelming, start with only:

```
my-project/
│
├── PRD.md
├── RULES.md
├── TASKS.md
├── README.md
├── .env.example
│
└── src/
```

Then add:

ARCHITECTURE.md
DESIGN.md
TEST_PLAN.md
SECURITY.md
DECISIONS.md
MEMORY.md

as your project becomes more complex.

---

40. The Professional Vibe Coding Loop

For every feature, follow this exact loop:

1. READ
↓
2. UNDERSTAND
↓
3. PLAN
↓
4. IMPLEMENT
↓
5. TEST
↓
6. REVIEW
↓
7. FIX
↓
8. COMMIT
↓
9. UPDATE DOCUMENTATION

Then move to the next feature.

Vibe Coding: A Complete Beginner-to-Production Guide